SECTION A

Part 01:

Double-click (or enter) to edit

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv(r'/content/Sales Data Set New.csv')
```

```
data.head()
```

	Transaction_ID	Date	Product	Category	Region	Price_Unit	Quantity	Customer_Rating
0	10001	2023-04-13 19:28:00	Headphones	Accessories	West	123.38	9	3.266932
1	10002	2023-09-28 10:07:00	Monitor	Electronics	South	331.06	4	1.819384
2	10003	2023-07-08 20:38:00	Tablet	Mobile	West	381.35	6	1.780775
3	10004	2023-05-02 18:22:00	Smartwatch	Mobile	Central	279.49	7	2.735333
4	10005	2023-11-27 10:23:00	Keyboard	Accessories	North	16.86	4	2.931962

Next steps: Generate code with data | New interactive sheet

```
df.describe()
```

	Transaction_ID	Price_Unit	Quantity	Customer_Rating
count	5000.000000	4750.000000	5000.000000	4600.000000
mean	12500.500000	501.509766	5.068000	2.978599
std	1443.520003	486.747834	2.571206	1.155197
min	10001.000000	-286.970000	1.000000	1.000662
25%	11250.750000	198.055000	3.000000	1.979616
50%	12500.500000	351.205000	5.000000	2.978769
75%	13750.250000	744.015000	7.000000	3.974583
max	15000.000000	6166.840000	9.000000	4.999599

Check for missing values

```
print("Missing values per column:")
print(df.isnull().sum())
```

```
Missing values per column:
Transaction_ID       0
Date                 0
Product              0
Category             0
Region               0
Price_Unit         250
Quantity             0
Customer_Rating    400
dtype: int64
```

Handling Missing Values

```
# We use median for Price_Unit to avoid outlier influence
df['Price_Unit'] = df['Price_Unit'].fillna(df['Price_Unit'].median())
```

We use mean for Customer_Rating to get the average sentiment

```
df['Customer_Rating'] = df['Customer_Rating'].fillna(df['Customer_Rating'].mean())

print("\nMissing values after processing:", df.isnull().sum().sum())
```

```
Missing values after processing: 0
```

PART 2

Calculating the Interquartile Range (IQR) for Price_Unit

```
Q1 = df['Price_Unit'].quantile(0.25)
Q3 = df['Price_Unit'].quantile(0.75)
IQR = Q3 - Q1
```

Defining the bounds for outliers

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

Identifying the outliers in the dataset

```
outliers = df[(df['Price_Unit'] < lower_bound) | (df['Price_Unit'] > upper_bound)]

print(f"Lower Bound: {lower_bound}")
print(f"Upper Bound: {upper_bound}")
print(f"Number of outliers detected: {len(outliers)}")
print(outliers[['Transaction_ID', 'Product', 'Price_Unit']].head())
```

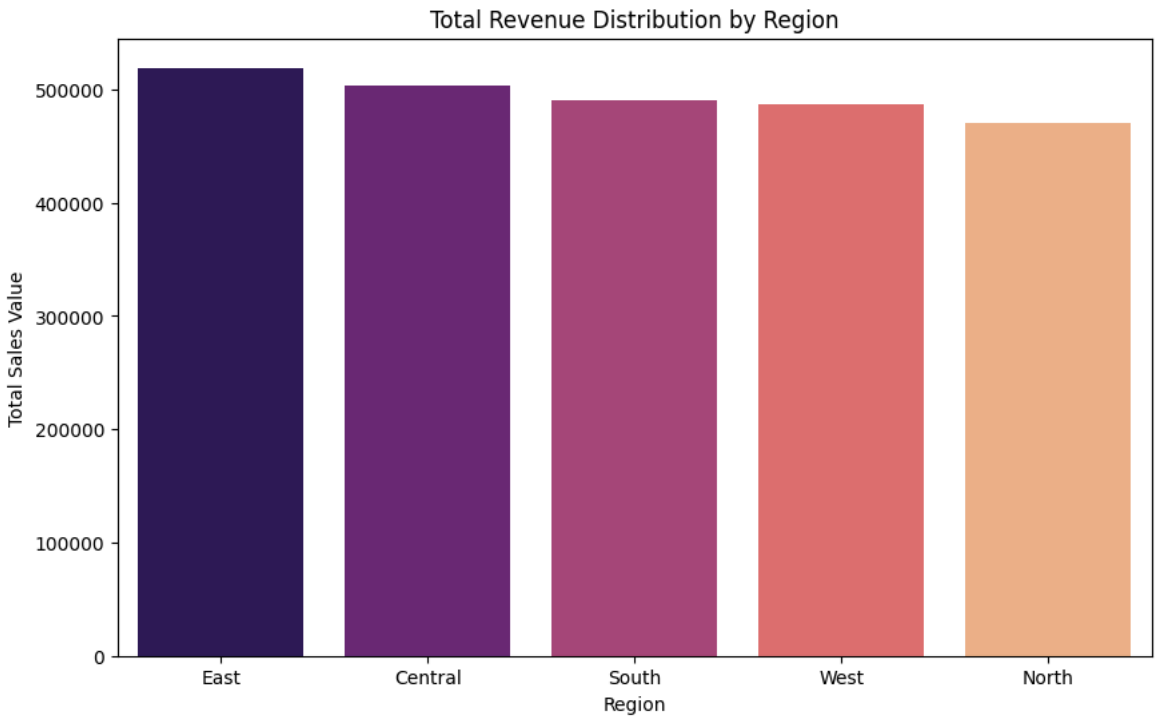
```
Lower Bound: -546.655
Upper Bound: 1461.805
Number of outliers detected: 54
     Transaction_ID  Product  Price_Unit
69            10070   Laptop     5834.44
96            10097   Laptop     6116.09
119           10120   Tablet     1897.39
285           10286   Camera     3678.21
457           10458  Monitor     1812.36
```

SECTION B

PART 01: I am using a Bar Chart to compare the total revenue generated across different regions.

```
# ANALYZING SALES DISTRIBUTION BY REGION

# Grouping data by Region Means That Region Sales Will Be Sum
region_sales = df.groupby('Region')['Price_Unit'].sum().sort_values(ascending=False)

# Visualization
plt.figure(figsize=(10, 6))
sns.barplot(x=region_sales.index, y=region_sales.values, palette='magma')
plt.title('Total Revenue Distribution by Region')
plt.xlabel('Region')
plt.ylabel('Total Sales Value')
plt.show()
```

```
/tmp/ipykernel_1111/2498996533.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.barplot(x=region_sales.index, y=region_sales.values, palette='magma')
```

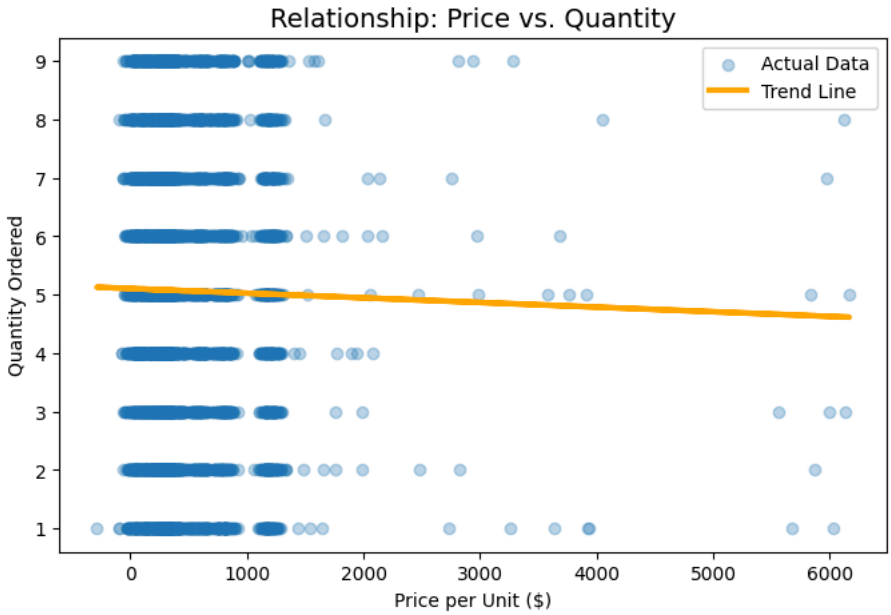


Double-click (or enter) to edit

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import linregress

# 1. Clean Data
df['Price_Unit'] = df['Price_Unit'].fillna(df['Price_Unit'].median())
df['Customer_Rating'] = df['Customer_Rating'].fillna(df['Customer_Rating'].mean())

# 2. Measures of Central Tendency and Dispersion
# .describe() gives us mean, std, min, max, and quartiles automatically
stats = df['Price_Unit'].describe()
iqr = stats['75%'] - stats['25%']
mode_val = df['Price_Unit'].mode()[0]

print("Statistical Summary ")
print(f"Mean (Average): {stats['mean']:.2f}")
print(f"Median (Middle): {stats['50%']:.2f}")
print(f"Mode (Most Frequent): {mode_val:.2f}")
print(f"Standard Deviation (Spread): {stats['std']:.2f}")
print(f"IQR (Middle 50% range): {iqr:.2f}")

# 3. Simple Regression Analysis
# Identifying the relationship between Price (X) and Quantity (Y)
X = df['Price_Unit']
Y = df['Quantity']

slope, intercept, r_value, p_value, std_err = linregress(X, Y)
line_of_best_fit = slope * X + intercept

print(f"\n--- Regression Results ---")
print(f"Slope: {slope:.4f}")
print(f"R-squared (Accuracy): {r_value**2:.4f}")

# 4. Visualization
plt.figure(figsize=(8, 5))
plt.scatter(X, Y, alpha=0.3, label='Actual Data') # Scatter plot
plt.plot(X, line_of_best_fit, color='orange', linewidth=3, label='Trend Line') # Regression line
plt.title("Relationship: Price vs. Quantity", fontsize=14)
```

```
plt.xlabel("Price per Unit ($)")
plt.ylabel("Quantity Ordered")
plt.legend()
plt.show()
```

```
Statistical Summary
Mean (Average): 493.99
Median (Middle): 351.21
Mode (Most Frequent): 351.21
Standard Deviation (Spread): 475.55
IQR (Middle 50% range): 502.12

--- Regression Results ---
Slope: -0.0001
R-squared (Accuracy): 0.0002
```



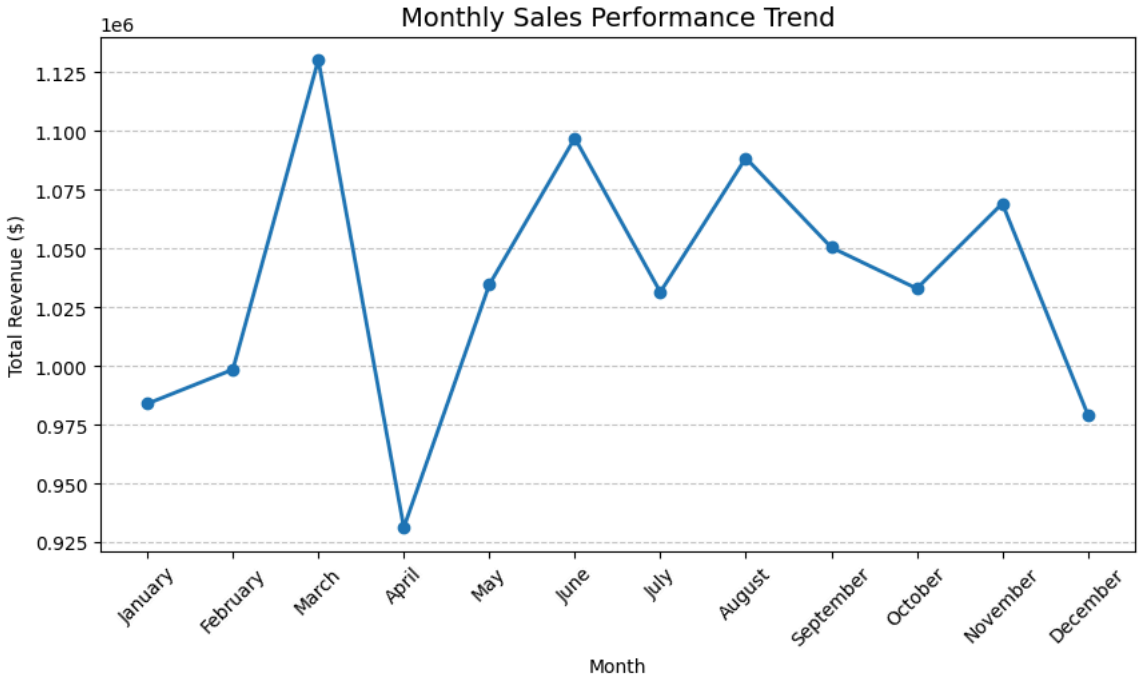
Part 2:

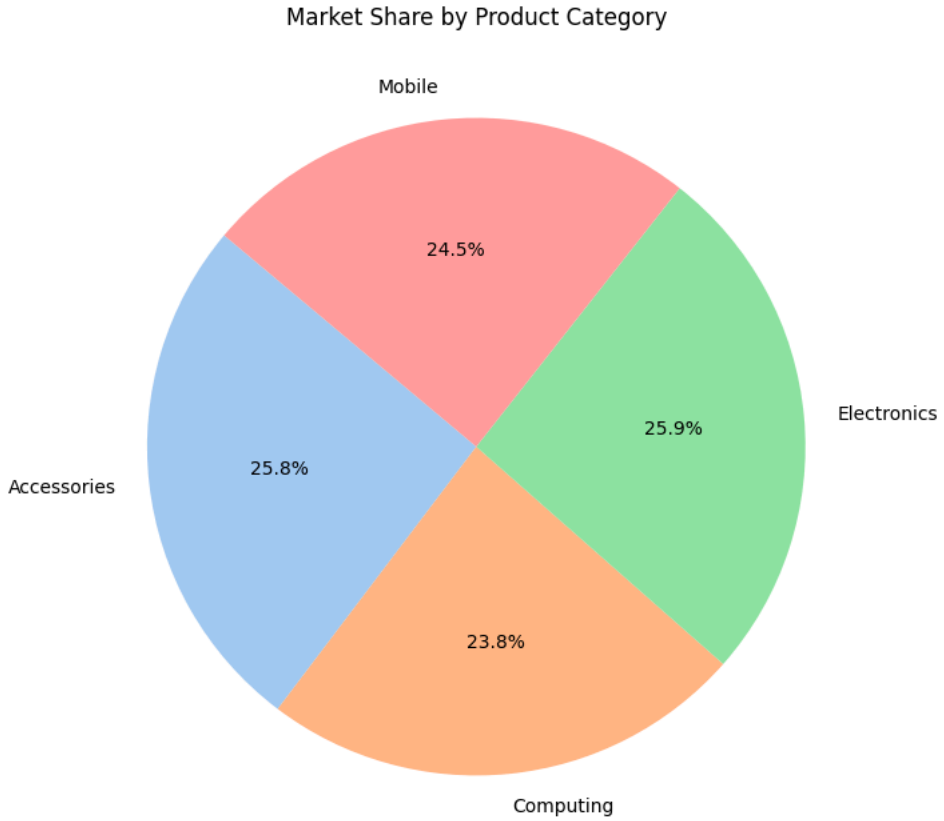
Logic: I am using a Pie Chart to show the proportion of each category within the total inventory sold. This provides a image or visual of the market share for each product type.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Prepare Data
df['Price_Unit'] = df['Price_Unit'].fillna(df['Price_Unit'].median())

# Calculate Total Sales for line chart
df['Total_Sales'] = df['Price_Unit'] * df['Quantity']

# Convert Date to datetime and extract Month for tracking progress
df['Date'] = pd.to_datetime(df['Date'])
df['Month'] = df['Date'].dt.month_name()

# Ensure that month are in order
month_order = ['January', 'February', 'March', 'April', 'May', 'June',
               'July', 'August', 'September', 'October', 'November', 'December']
df['Month'] = pd.Categorical(df['Month'], categories=month_order, ordered=True)
monthly_progress = df.groupby('Month')['Total_Sales'].sum()

# LINE CHART
plt.figure(figsize=(10, 5))
plt.plot(monthly_progress.index, monthly_progress.values, marker='o', color='tab:blue', linewidth=2)
plt.title('Monthly Sales Performance Trend', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Total Revenue ($)')
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
plt.xticks(rotation=45)
plt.show()
```

```
# PIE CHART
category_qty = df.groupby('Category')['Quantity'].sum()

plt.figure(figsize=(8, 8))
plt.pie(category_qty, labels=category_qty.index, autopct='%1.1f%%', startangle=140, colors=sns.color_palette('pastel'))
plt.title('Market Share by Product Category')
plt.show()
```

```
/tmp/ipykernel_1111/1098093274.py:19: FutureWarning: The default of observed=False is deprecated and will be changed to True in
  monthly_progress = df.groupby('Month')['Total_Sales'].sum()
```





SECTION C

PART 15: I am creating a model to predict Total Sales based on the product's Price_Unit and Quantity.

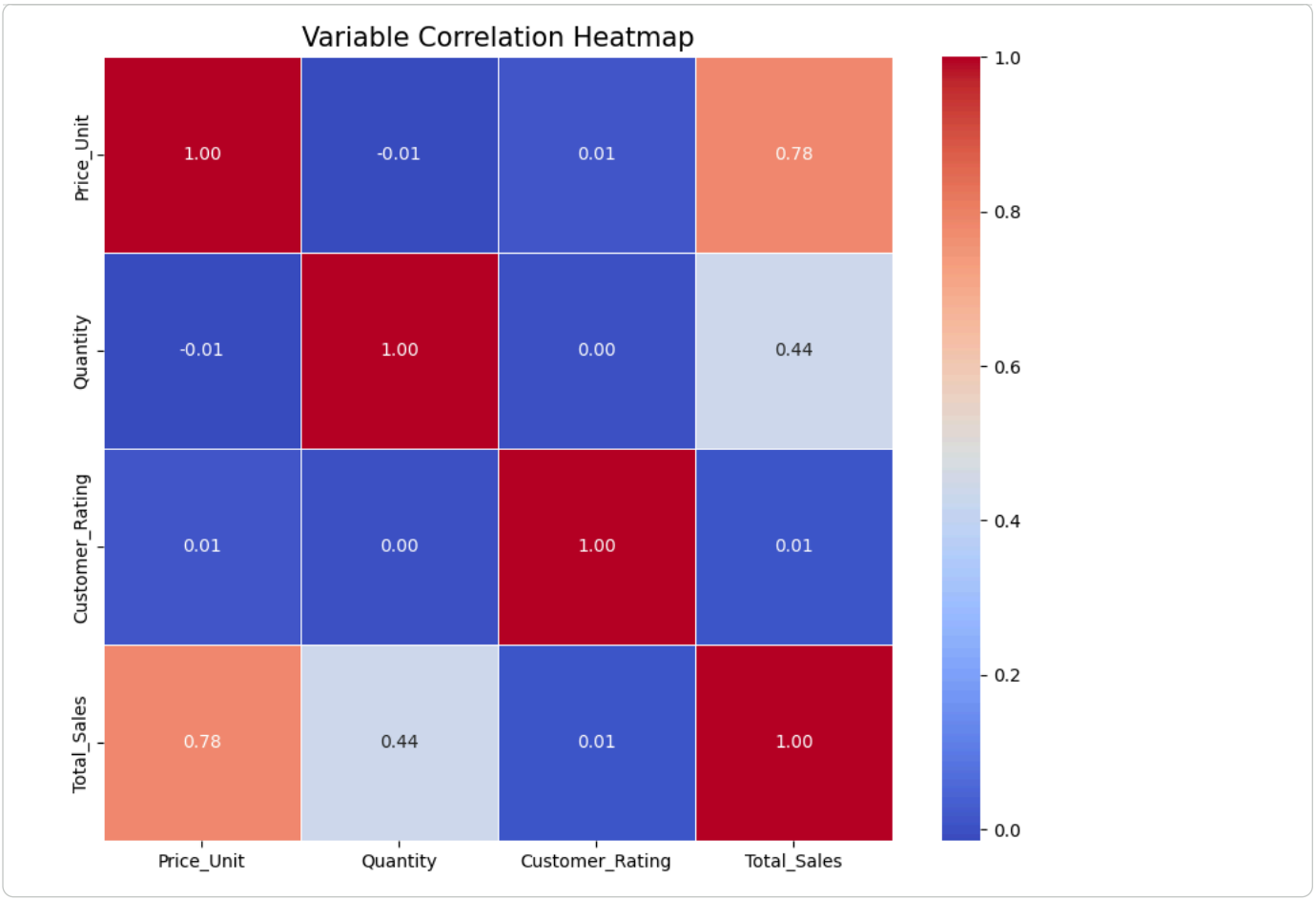
Split our data into a **Training Set** (to teach the model) and a **Testing Set** (to check its accuracy). This ensures the model can handle new, unseen data.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Prepare Features (X) and Target (y)
# We use Price and Quantity to predict Total Sales
X = df[['Price_Unit', 'Quantity']]
y = df['Price_Unit'] * df['Quantity']  # This is our actual 'Total Sales'

# 2. Split data: 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Initialize and Train the Model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Make Predictions
predictions = model.predict(X_test)

# 5. Evaluate Accuracy
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print(f"Model Accuracy (R-squared): {r2:.4f}")
print(f"Mean Squared Error: {mse:.2f}")
```

```
Model Accuracy (R-squared): 0.8482
Mean Squared Error: 1964079.29
```

Part 16: I am using a Correlation Matrix. A value of 1.0 means perfect positive correlation, while 0 means no relationship at all.

```
import seaborn as sns

# 1. Calculate the correlation matrix for numerical columns
# We look at Price, Quantity, Rating, and Total Sales
correlation_matrix = df[['Price_Unit', 'Quantity', 'Customer_Rating', 'Total_Sales']].corr()

# 2. Visualize with a Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Variable Correlation Heatmap', fontsize=15)
plt.show()
```



Explanation:

If **Price_Unit** and **Total_Sales** have a high positive correlation (e.g., 0.85), it tells us that our revenue is primarily driven by selling expensive items rather than selling a high volume of cheap ones.

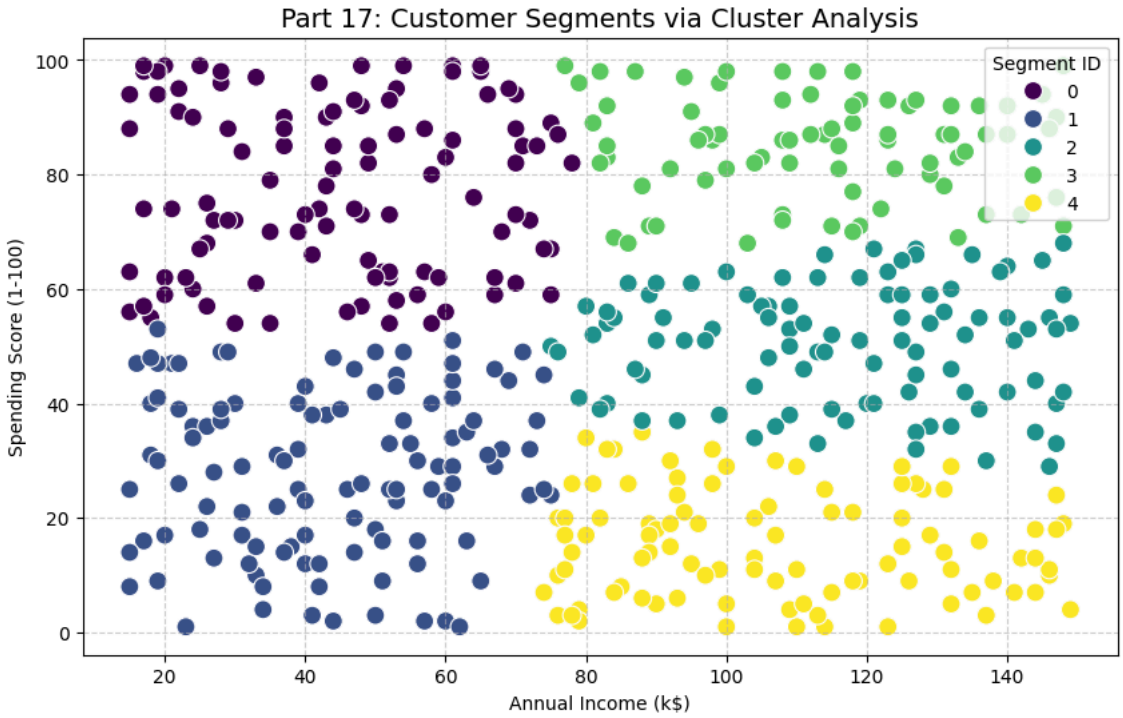
If **Customer_Rating** has a low correlation with sales, it might suggest that customers buy these products out of necessity rather than based on reviews.

Section D

Part 17: This script uses K-Means Clustering to group customers based on their Income and Spending Score. This helps businesses identify which groups to target with specific marketing campaigns.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Load the dataset
df = pd.read_csv('Financial_Data.csv')

# 2. Select Features for Clustering
# We focus on Annual Income and Spending Score to find patterns
X = df[['Annual_Income_k', 'Spending_Score']]

# 3. Data Scaling (Crucial for Cluster Analysis)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Initialize and Apply K-Means
# We choose 5 clusters to represent different customer types
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df['Customer_Segment'] = kmeans.fit_predict(X_scaled)
```

```
# 5. Visualization
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Annual_Income_k', y='Spending_Score',
                hue='Customer_Segment', palette='viridis', s=100)

plt.title('Part 17: Customer Segments via Cluster Analysis', fontsize=14)
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(title='Segment ID')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# Print the segment counts
print("Customers per Segment:")
print(df['Customer_Segment'].value_counts())
```




```
Customers per Segment:
Customer_Segment
1    116
0    109
2    100
4     95
3     80
Name: count, dtype: int64
```

Part 19: This script develops a Random Forest machine learning model to predict if a transaction is fraudulent based on the customer's age, income, and the transaction amount.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# 1. Load the dataset
df = pd.read_csv('Financial_Data.csv')

# 2. Define Features (X) and Target (y)
X = df[['Age', 'Annual_Income_k', 'Transaction_Amount']]
y = df['Is_Fraud']

# 3. Split the Data (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Build the Machine Learning Model
# Random Forest is highly effective for fraud detection
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
model.fit(X_train, y_train)

# 5. Evaluate the Model
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("--- Part 19: Fraud Detection Performance ---")
print(f"Overall Model Accuracy: {accuracy * 100:.2f}%")
print("\nDetailed Classification Report:")
print(classification_report(y_test, predictions))

# Feature Importance: See what matters most in detecting fraud
importance = pd.Series(model.feature_importances_, index=X.columns)
print("\nKey Factors in Fraud Detection:")
print(importance.sort_values(ascending=False))
```

```
--- Part 19: Fraud Detection Performance ---
Overall Model Accuracy: 92.00%

Detailed Classification Report:
              precision    recall  f1-score   support

           0       0.92      1.00      0.96        92
           1       0.00      0.00      0.00         8
```